

POC SPEC V2 — Catalyst Core-Loop Prototype ("Does the flip work on reload?")

Version: 2.0 | Status: Draft for 2-day build | Sources: docs/v2/PRD.md, docs/v2/TECH_SPEC.md, docs/v2/USE_CASES.md

Timebox: 2 days, strict. If it takes longer, the POC is already telling you the scope is wrong — cut, don't extend.

> Change from V1 (POC v1.0): SSE is gone. The loop under test is toggle flag -> reload app -> new value + reason -> evaluation sticky. Same model that V2 ships with (fetch-on-load), so the POC and the real build share the exact propagation semantics.

Goal	Prove toggle -> SDK picks it up on next load -> evaluation sticky -> offline never crashes
Gateway	Express, in-memory flag, /bootstrap with ETag/304, no SSE
Source	docs/v2/PRD.md, docs/v2/TECH_SPEC.md, docs/v2/USE_CASES.md

This PDF was exported from v2/POC.md. The Markdown file remains the source of truth; this PDF is the shareable snapshot for review.

0. Goal + Hypothesis

Goal: prove the single riskiest loop of Catalyst end to end: toggle flag in dashboard -> SDK picks it up on next load -> evaluation is sticky -> offline never crashes.

Hypotheses under test:

#	Hypothesis	If false, then...
H-1	A dashboard toggle is observed on the next app load (init -> bootstrap) and the app re-renders correctly	Bootstrap contract (TECH_SPEC v2 §5) is wrong; redesign the snapshot/ETag shape before V2 build
H-2	Percentage rollout is sticky per user (same user, same result, 1000 evals)	Hash approach (murmur + key) is broken; pick a new bucketing scheme before V1
H-3	init / isEnabled feels right inside a React component	SDK surface needs redesign before building the real JS + Python SDKs
H-4	Plain fetch bootstrap + ETag/304 is painless enough for a solo dev	Reconsider caching/version scheme before Wk3

Everything else (auth, multi-env, rules, audit, Python, Postgres, push) is deliberately excluded — see §4.

1. Features (P0 only — six items, no more)

- **F-1 Kill toggle API:** single hardcoded flag new-checkout. PUT /poc/flag {enabled, percentage} flips it, bumps version. GET /poc/flag reads it.
- **F-2 Deterministic evaluation:** POST /poc/evaluate {flagKey, context:{userId}} -> {value, reason, version}. Order: kill OFF -> defaultValue (disabled); else rollout `murmur(flagKey:userId) % 100 < percentage -> true (rollout); else defaultValue (default)`. No rules in POC.
- **F-3 Bootstrap fetch:** GET /poc/bootstrap -> {version, flags:[{key, enabled, percentage, defaultValue, version}]} honoring If-None-Match -> 304 when unchanged. This is what the SDK calls on load.
- **F-4 Minimal JS SDK (~80 lines, zero deps):** `init({baseUrl}) -> bootstrap once, isEnabled(key, ctx)` from memory, cache in `localStorage` so a reload still works — even when toggles changed offline. No SSE, no polling, no `onUpdate`.
- **F-5 Demo page (single demo.html, no build step):** shows NEW vs OLD checkout box, current value + reason + version, and the toggle-version it bootstrapped against. Reload the page to pick up a new toggle.
- **F-6 Stickiness script:** `node poc/check.js` evaluates one user 1000x (same result) plus 10k synthetic users at 50% (eyeball distribution ~ half/half).

2. Specification

Layout (throwaway folder; only the learnings carry into V2):

```
poc/
  server.js    # Express, in-memory flag, 4 endpoints
  hash.js     # murmur3_32, shared by server + SDK (copy, don't import — proves the port workflow)
  sdk.js      # browser SDK, fetch only, no deps
  demo.html   # imports sdk.js, two boxes + status line
  check.js    # stickiness + distribution sanity
```

API contract:

- 1 PUT /poc/flag {enabled: bool, percentage: 0-100} -> {key:"new-checkout", enabled, percentage, defaultValue:false, version} (400 on bad %).
- 2 GET /poc/flag -> same shape (lets curl verify state).
- 3 POST /poc/evaluate {flagKey, context:{userId?}} -> {value, reason: "disabled|rollout|default", version}. Missing userId -> skip rollout -> defaultValue.
- 4 GET /poc/bootstrap -> {version, flags:[...]}. If If-None-Match matches current version -> 304 empty body (lets the SDK prove the jump-to-latest path and the no-change path).

SDK contract:

```
await poc.init({ baseUrl: 'http://localhost:3001' });
poc.isEnabled('new-checkout', { userId: 'u123' }); // bool, never throws
// reload the page (or re-init) to observe a toggle
```

Run:

```
node poc/server.js          # :3001
open poc/demo.html         # or npx serve .
curl -X PUT :3001/poc/flag -H 'Content-Type: application/json' -d '{"enabled":false,"percentage":50}'
node poc/check.js
```

3. Validation protocol (the actual point)

- 1 Open `demo.html` twice (two users: `u123`, `u456`). Both show value + reason.
- 2 `curl` kill OFF -> reload both pages -> both flip to OLD, reason `disabled`. Film the before/after — it becomes your first demo clip.
- 3 Kill ON, percentage: 1 -> reload pages ~10x each: each user stable (sticky), users may differ (bucketing). Reason rollout vs default.
- 4 Kill the server mid-session -> reload -> page still renders from `localStorage` cache, version stale but never crashes. Restart server -> reload -> fresh values.
- 5 `node poc/check.js` -> 1000/1000 identical; 10k-user split within eyeball range of 50/50.

Success gates (all must pass):

- G-1: toggle -> reload flips the page with correct reason, twice in a row.
- G-2: 1000/1000 sticky; distribution sane.
- G-3: no-Throw: dead server + reload never breaks the page.
- G-4: verdict written down: GO to V2, or NO-GO with the falsified hypothesis (H-1..H-4) and what changes.

4. Non-goals (explicitly OUT — do not build)

Auth/JWT, `sdk_key`, multi-env, targeting rules, audit log, Postgres/Redis, Python SDK, React dashboard, drag-reorder, playground, key rotation, SSE/polling/WebSockets. Server restart loses state (in-memory) — accepted; persistence is a V2 problem. If you catch yourself adding any of these, stop: that is V2 scope creep wearing a POC costume.

5. Handoff to V2 (what survives the POC)

- **Keeps:** eval order, murmur seed + `flagKey:userId` format, bootstrap shape (`{version, flags}`) + ETag/304, SDK surface (`init/isEnabled`), fetch-on-load refresh semantics, the demo-reload script as V2's first acceptance test.
- **Throws away:** `poc/` code itself (rewritten against Postgres + auth in Wk1–Wk3), in-memory store, single-flag hardcoding.
- **Decides:** H-1 verdict locks the bootstrap contract in TECH_SPEC v2 §5; H-3 verdict locks the SDK chapter before a line of the real SDK is written.